CS781 Capstone Project: In-House Legal Intelligence System

Nitin Maheshwari
230711
nitinm23@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

I built a compact legal language modelling pipeline and adapted it to entity extraction, rhetorical roles, judgment prediction, statute identification, summarization and prior-case retrieval. Four foundation architectures fit within the 50M–300M parameter limit. A 280.8M-parameter dense decoder produced the best recorded submission in every phase, including 0.7935 judgment macro-F1 and 0.1855 retrieval micro-F1@7. This was not a matched-budget architecture comparison: its later foundation checkpoint had substantially more training than the alternatives. The clearest trade-offs concerned restricted document coverage, adapter efficiency and incomplete experimental coverage. I separate finished uploads from failed attempts, local metrics from competition scores, and implemented experiments from unexecuted tuning proposals. An implementation diagnostic also identifies padding sensitivity in the encoder path.

1 Competition Result

Codabench username: `n_230711` **Roll number:** 230711

Table 1: Best finished submission visible in the supplied results document. Ranks are the values reported in that document. The count includes uploads with a displayed score; individual statuses are listed in Appendix A.

Phase	Task / primary metric	Score	Rank	Scored
1	LM continuation / BERTScore F1	0.8273	106	5
2	Legal NER / strict macro-F1	0.3703	113	4
3	Rhetorical roles / macro-F1	0.5146	50	5
4	Judgment prediction / macro-F1	0.7935	22	4
5	Statute identification / macro-F1	0.3517	92	4
6	Summarization / BERTScore F1	0.8153	119	4
7	Prior-case retrieval / micro-F1@7	0.1855	126	3

Public GitHub URL: https://github.com/maheshwari-nitin/cs781

2 Introduction and Problem Description

The project considers a legal language system that can be hosted without sending case material to an external prediction service. The practical difficulty is not simply producing fluent legal text. A useful system must recover exact entity boundaries, distinguish a court's reasoning from quoted precedent, predict several applicable statutes, summarize a holding and retrieve relevant earlier cases. These tasks need different outputs and different error checks.

The course fixes a 50M–300M foundation-model budget and requires training the foundation without loading pretrained language-model weights. I explored a dense decoder, an encoder–decoder, a recurrent-attention hybrid and a sparse-expert decoder. Later phases reuse those foundations in separate adapted models. The tasks follow the course benchmark derived from Indian legal text understanding and reasoning [1, 2]. Table 2 states the evaluation contract.

Table 2: Task contracts. Every phase preserves the original document/query identifiers.

P	Input	Required output	Main metric
1	512-word prompt	128-word continuation	BERTScore F1
2	Case text	Typed character spans	Strict macro-F1
3	Sentence sequence	One role per sentence	Macro-F1
4	Judgment text	Binary outcome	Macro-F1
5	Case text	Set of statute IDs	Macro-F1
6	Legal document	Generated summary	BERTScore F1
7	Query and split pool	Seven ranked case IDs	Micro-F1@7

The project examines how a compact legal language model can be adapted to tasks with different prediction requirements. I compare foundation models and adaptation methods across exact-span extraction, classification, generation and retrieval, with particular attention to the information retained from long judgments. The experiments assess whether updating the full backbone or a small set of adapter parameters gives the better task result, and identify failures that aggregate scores can hide, such as repeated summary sentences and irrelevant retrieved cases.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

The source is the official `Exploration-Lab/CS-781-Capstone` release. Phase 1 samples documents from Acts, High Courts, Supreme Court and Tribunals; the preparation recipe selects 10% of each source and holds out 0.5% of the selected documents. A separate full-data recipe preserves that holdout. These are source-code settings: retained per-source counts and actual consumed-token logs are unavailable. NER uses released folds, rhetorical roles use CL/IT train–dev files, and classification/retrieval use their official splits. Summarization has no public dev file, so it uses a document holdout from training.

3.2 Corpus statistics that actually informed design

Table 3 gives counts present in executed notebook outputs. There are 100 observed statute IDs, 0–99, with mean unique-label counts of 3.70 in train and 3.92 in dev. Visible support counts already show imbalance: statute 4 occurs 341 times and statute 97 occurs 7,880 times in train. These examples are not asserted to be the global minimum and maximum; the full frequency table was truncated on display.

Table 3: Recorded dataset sizes. PCR columns are query count / candidate count; summarization's holdout comes from the 7,028 usable training records. Early-phase counts were not retained.

Task	Training	Dev / holdout	Test
Judgment prediction	37,387	1,988	1,517
Statute identification	42,750	10,181	13,019
Summarization	6,678	350 (32 scored)	100
Prior-case retrieval	827 / 4,320	118 / 1,023	237 / 1,727

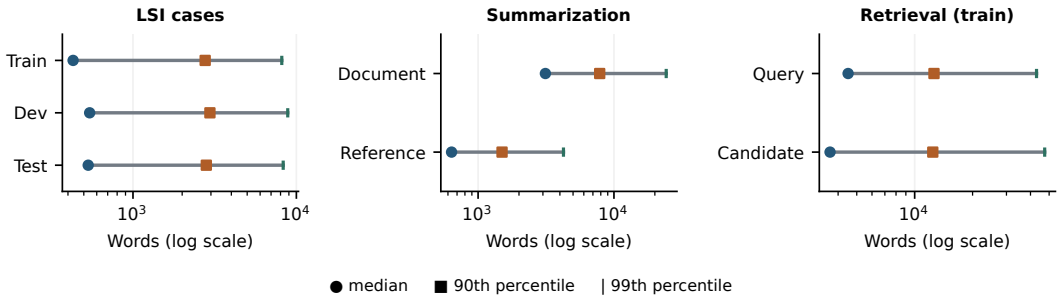


Figure 1: Original plot of recorded word-length percentiles, not reconstructed histograms. The source is the data-overview output in the LSI, summarization and PCR notebooks.

Judgment training has 22,067 class-0 and 15,320 class-1 examples, approximately 59%/41%. Legal cases are much longer than the models' windows: retrieval training queries have median 4,508 words, and one candidate reaches 410,597 words. In summarization, the marginal medians are 3,129 document words and 638 reference words. Their ratio is not the median paired compression ratio. Two records, IDs 299 and 4799, have missing/blank summaries and are excluded before the holdout split.

3.3 Preprocessing

NER retains the raw text because normalizing whitespace would move gold character offsets. Byte-level BPE offsets are trimmed only at a token's leading whitespace before alignment. Rhetorical roles retain the supplied sentence boundaries; they are not resegmented. Classification, summarization and retrieval may collapse whitespace before tokenization. Duplicate statute annotations are converted to sets. Empty retrieval text keeps its ID and receives a BOS representation rather than silently shrinking a pool. All loaders check identifier/schema consistency; test reference fields are not training inputs.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

I implemented four foundation families, summarized in Table 4. A follows a modern dense causal-decoder design [3]: grouped-query attention, per-head query/key normalization, rotary positions, RMS normalization and SwiGLU [4, 5, 6, 7]. Each block applies attention and MLP updates through pre-norm residual paths. Embedding and output weights are tied. B has bidirectional encoder blocks and causal decoder blocks with cross-attention, following the encoder–decoder organization of T5 [8]. C alternates three gated DeltaNet blocks with one gated attention block [9]. D uses latent KV compression and shared/routed feed-forward experts, inspired by DeepSeek [10]; it also exposes an optional multi-token objective.

Table 4: Implemented defaults and observed parameter counts. D's base count is recovered from its logged 100-label classifier count after subtracting the 76,900-parameter head.

Model	Structure	Width	Parameters (M)	Attention / FFN
A	22 decoder blocks	1,024	280.805	16 Q / 4 KV; FFN 2,816
B	10 encoder + 10 decoder	1,024	284.483	GQA + cross-attention
C	24 hybrid blocks	1,024	291.581	3:1 GDN/attention; FFN 1,920
D	16 MLA–MoE blocks	768	233.974	12 routed, 1 shared; top-2

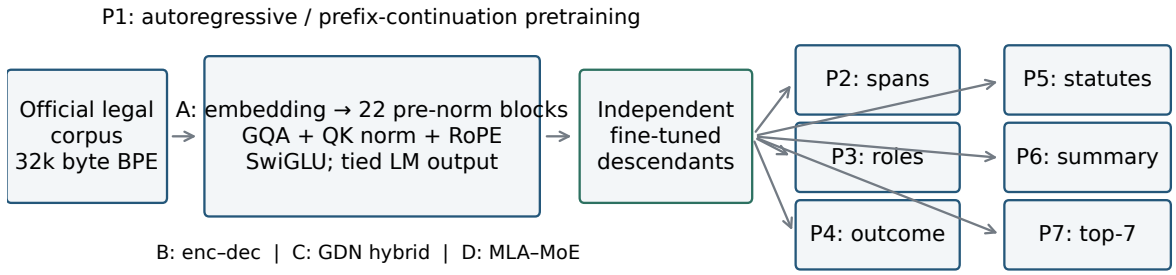


Figure 2: Implementation structure. Task models are separately adapted descendants; NER or role outputs are not automatically fed into the judgment, statute, summary or retrieval systems.

For A, the unique-parameter calculation is explicit:

$$N = 32{,}000 \times 1{,}024 + 22 \times 11{,}274{,}368 + 1{,}024 = 280{,}805{,}120.$$

The code also counts unique parameter tensors so tying is not double-counted. All four implementations satisfy the stated parameter interval. The 32k byte-level BPE handles names and citations without an out-of-vocabulary replacement requirement. Decoder pretraining predicts every next token; B predicts a 256-token continuation conditioned on a 768-token prefix. Phase-1 generation targets 128 words, using the archived greedy recipe with repetition penalty 1.1; the exact winning dump settings were not retained.

4.2 Transfer protocol (Phases 2–7)

Task models load a saved Phase-1 checkpoint and add a task head where needed. Feature extraction uses an explicit hidden-state method, B's encoder, or a hook immediately before the tied LM output. The hook avoids materializing vocabulary logits for classification and retrieval. Full adaptation trains the used backbone parameters; partial adaptation trains the last feature blocks and final norms. In B's classifier and retriever, unused decoder parameters are frozen. This saves optimizer work but does not physically remove the decoder from the stored model.

The low-rank alternatives retain frozen base projections. In conventional column-vector notation,

$$W_{\text{LoRA}} = W_0 + sBA, \qquad W_{\text{LoRMA}} = (I + sBA)W_0, \qquad s = \alpha/r.$$

LoRA adds an input-space low-rank update [11]; the implemented LoRMA wrapper applies an output-space multiplicative update, keeps the original bias separate and initializes one factor to zero [12]. This describes the repository implementation, not every variant in the LoRMA paper. QLoRA uses NF4 frozen base projections with floating-point adapters; tied embeddings/output remain unquantized [13]. Its implementation is restricted to A.

4.3 Phase 2 - Legal NER

NER is token classification with BIO labels for the 12 entity types. Gold character intervals are aligned to tokenizer offsets; predicted BIO sequences are reconstructed into exclusive-end character spans in the unchanged document. Overlapping windows use a deterministic owner rule: more left context for causal models, and more central placement for bidirectional encoders. Argmax and constrained BIO Viterbi decoding are implemented. An oracle reconstruction check exposes the ceiling imposed by token boundaries and overlapping spans. Its measured value was not retained, so I do not report one.

4.4 Phase 3 - Rhetorical role prediction

The role classifier pools the token interval of each supplied sentence into 13 label logits. A document mode supplies neighboring sentences through overlapping windows; a sentence mode is also available. CL and IT are trained jointly in the archived recipe and evaluated by domain as well as jointly. A long sentence is capped to fit inside at least one window. No document-level CRF is used. Synthetic-generation utilities exist in the working repository, but I confirmed that their documents were not used in training.

4.5 Phase 4 - Court judgment prediction

The classifier reads four 512-token chunks from the judgment's end, mean-pools tokens and chunks, and predicts two logits with cross-entropy. The recorded runs use a fixed 0.5 decision threshold; balanced training weights address the observed 59%/41% class split in the saved KD configuration. Mean, max and learned chunk attention are implemented, but the local full/LoRMA/KD rows change more than one factor. They are variant comparisons, not isolated tests of chunk aggregation.

4.6 Phase 5 - Legal statute identification

LSI reuses the document classifier with one output per observed statute. Independent sigmoids are necessary: mutually exclusive softmax predictions would contradict documents with several correct IDs. Training minimizes

$$\mathcal{L}_{\text{LSI}} = -\frac{1}{BK}\sum_{b,k}\{y_{bk}\log\sigma(z_{bk}) + (1-y_{bk})\log[1-\sigma(z_{bk})]\}.$$

The recorded configuration uses four spread 512-token chunks and a fixed 0.5 threshold. Optional positive weights and dev-calibrated global/per-label thresholds are implemented but are not credited as completed improvements. The label-to-column map is persisted even though the observed IDs happen to be contiguous. KD for D uses independent Bernoulli teacher targets rather than a categorical distribution over statutes. Its student is genuinely smaller in total backbone parameters than A.

4.7 Phase 6 - Abstractive summarization

Decoder models receive document markers, selected source tokens and the target summary; prompt labels are masked. B uses encoder inputs and a BOS-shifted target in its decoder. The objective is

$$\mathcal{L}_{\text{summ}} = -\sum_{t=1}^{T_y}\log p_\theta(y_t \mid x, y_{<t}),$$

including the end token. The saved recipe budgets 640 source tokens, including markers, and 256 target tokens. Reference targets exceeding that budget are truncated before appending EOT. Inference permits EOT after 16 generated tokens and caps generation at 256. It does not inherit Phase 1's fixed word count. The recorded runs use greedy decoding without a repetition penalty or n-gram ban. A reproducible holdout groups duplicate normalized documents together; metrics are computed on a saved subset of 32 of the 350 held-out documents. RoBERTa-large supplies BERTScore only, never training or teacher targets [14]. The local scorer hash is retained in the appendix.

4.8 Phase 7 - Prior Case Retrieval (PCR)

A shared encoder pools tokens and chunks, then L2-normalizes the document vector. Cosine similarity provides the ranking score. The recorded run selects first/last context totaling two 256-token chunks. Each training query uses up to two positives and four fixed random negatives from its training pool; all known positives are excluded from negatives. With positive set P_q and sampled set C_q,

$$\mathcal{L}_q = -\frac{1}{|P_q|}\sum_{p\in P_q}\log\frac{\exp(s(q,p)/\tau)}{\sum_{c\in C_q}\exp(s(q,c)/\tau)}, \qquad \tau = 0.05.$$

Dev/test search uses the complete corresponding candidate pool, with no mixing across splits. Ten training queries have no positives and are skipped; all 118 dev and 237 test queries are retained. Retrieval KD and longer-training candidates are implemented but have no completed result in the supplied record. This distinction matters when auditing the mandatory KD requirement.

5 Experiments and Implementation Details

5.1 Training protocol

Only free Kaggle compute was used for training. Under this constraint, I could complete the planned full pretraining run only for model A; the other foundations were adapted from partially pretrained checkpoints. This allowed me to implement and evaluate all seven task pipelines, but the resulting comparison reflects both architecture and pretraining progress. AdamW, gradient clipping and fixed random seeds were used throughout. Phase 1 uses warmup–stable–decay, while the later loops use warmup followed by linear decay. Appendix H distinguishes saved settings from reconstructed configurations.

Table 5: Best-family recipes recoverable from source/output. * is an archived default, not a verified winning-run export. P4–P6 epoch/batch values shown in the executed last-run cells do not prove every earlier A run used those values. "NR" means not retained.

P	Context / representation	LR	Batch / accumulation	Budget
1	1,024-token blocks	$3\times10^{-4*}$	calibrated / 256 seq.*	NR actual tokens
2	1,024 / stride 768*	$5\times10^{-5*}$	$8\times1^{*}$	10 epochs*
3	document / 1,024*	$5\times10^{-5*}$	$8\times1^{*}$	10 epochs*
4	last 4×512	5×10^{-5}	8×1 (KD cell)	2 (KD cell)
5	spread 4×512	5×10^{-5}	8×4 (D cell)	1 (D cell)
6	source 640 / target 256	5×10^{-5}	1×8	1 (saved cell)
7	first/last 2×256	5×10^{-5}	32×1	3 epochs / 78 steps

Classification heads use a separate archived learning rate of 10^{-3}; the later recipes use weight decay 0.01 and clipping at 1.0. These settings explain the implementation without pretending that a notebook's final configuration is an immutable export of all preceding experiments.

5.2 Local evaluation

NER selects a training-document holdout; roles, judgment, LSI and PCR use official dev labels. Summarization uses the internal holdout described above. Local BERTScore settings are not guaranteed to match the competition scorer; ROUGE reports reference overlap [15]. Checkpoints are selected by the task metric, not test labels. Classification thresholds and decoding options are saved for inference. Phase 1 supports optimizer resume; the later snapshots are inference checkpoints rather than exact optimizer resumes. Original Phase-1 weights remain external to the submitted code archive.

5.3 Ablations and controlled comparisons

The archive supports comparisons, but few single-factor ablations. A's full-data descendant versus shorter-trained B/C changes both architecture and training. Full versus LoRMA changes the trainable parameter set, sometimes pooling and budget as well. The later 30-epoch A recipe, resampled/hard-negative options and larger retrieval context are unexecuted proposals in this record, not evidence of a gain. I report those limits rather than converting planned settings into experimental findings.

6 Results

Table 6: Best finished competition score per architecture family. The A column includes its better recorded variant. Dashes indicate no finished result in the supplied screenshot, not a zero score.

P	Metric	A	B	C	D
1	BERTScore F1	**0.8273**	0.8016	0.8144	0.8077
2	Strict macro-F1	**0.3703**	0.0006	0.2984	0.2918
3	Macro-F1	**0.5146**	0.3410	0.4092	0.3891
4	Macro-F1	**0.7935**	0.5914	—	—
5	Macro-F1	**0.3517**	0.0328	0.0951	0.0730
6	BERTScore F1	**0.8153**	0.7651	0.7680	—
7	Micro-F1@7	**0.1855**	0.0549	0.0732	—

A is the empirical winner of this implementation, but these rows do not establish an architectural cause. Its later foundation checkpoint has 16,800 pretraining updates, whereas the loaded B and C checkpoints record 600 and 200. A's validation tail loss is 1.2521, versus B's 2.9320 and C's 2.9727. That pre-existing gap is a serious confound before any task-specific adaptation begins. The second A rhetorical-role upload improves from 0.4837 to 0.5146; the retained record does not isolate the changed setting.

Table 7: Local A metrics and corresponding competition score. The evaluated populations differ; the summarization local score uses only 32 internal holdout documents. These are not paired test estimates.

P	Variant	Local primary	Competition	Local secondary metrics
4	full A	0.814897	0.7935	NR for this row
5	full A	0.345304	0.3517	micro-F1 .492551; subset .093311
6	A LoRA	0.826998	0.8153	R1 .301680; RL .197140
7	full A	0.192329	0.1855	P@7 .209443; R@7 .177801

The summarization comparison favors ordinary LoRA over QLoRA in this record. Both runs have 3,198,976 trainable parameters, while local BERTScore is 0.826998 for LoRA and 0.813496 for QLoRA. The corresponding competition scores, 0.8153 and 0.8094, follow the same ordering. This comparison supports selecting LoRA for the final summary model, although the retained run settings do not isolate quantization as the sole cause of the difference.

For PCR, A retrieves 173 relevant cases among 826 outputs, against 973 complete gold references: $F_1 = 2(173)/(826 + 973) = 0.192329$. C retrieves 73 and B 65. This calculation verifies the full-gold denominator rather than truncating each gold list to seven. Additional variant metrics are listed in Appendix B.

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

B is the clearest failed attempt: its NER score is 0.0006 and it trails A on every recorded phase. There are three concrete issues to separate. First, B received far fewer pretraining updates. Second, its prefix objective supervises only 256 positions per 1,024-token window, whereas A supervises every next-token position. Third, downstream feature calls pass no padding mask to B's bidirectional encoder. Right-padding tokens can therefore affect otherwise valid token representations. These observations support investigating B's training and interface, not dismissing encoder–decoder models in general.

A controlled implementation check uses two-layer, width-32 random models, not competition weights. Appending eight PAD tokens changes B's original-token features by relative L2 0.3133; A's change is 5.77×10^{-8}. This confirms padding sensitivity in that code path. It does not quantify how much of B's final NER failure it caused. Appendix F gives the exact test and its limits.

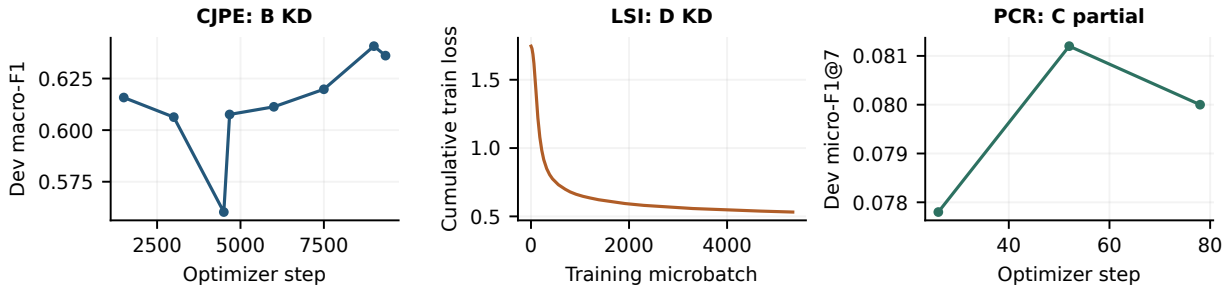


Figure 3: Only retained trajectories are plotted: judgment B-KD dev scores, LSI D-KD cumulative training loss, and retrieval C dev scores. The LSI trace is a running average, not independent minibatch losses. No missing A learning curve is reconstructed.

The judgment KD trace peaks at dev macro-F1 0.6407 at step 9,000, then ends at 0.6361. Keeping the best checkpoint matters, but the failed competition upload prevents calling it a valid competition improvement. In LSI, low macro-F1 alongside higher micro-F1 points to unequal label performance; the missing full per-label output prevents naming all weak statutes. The fixed threshold was not validated against the implemented calibration alternatives.

The summarization preview contains repeated sentences rather than merely low overlap. One test preview repeats "He was found guilty of the charge framed against him." 17 times in 229 words. The settings use no repetition penalty and permit 256 new tokens; a high BERTScore does not remove this defect. PCR also exposes a concrete retrieval error: query `0000035468` ranks its own ID first, while the displayed dev row has zero hits against 15 references. Self-matching consumes a slot in this example. The same pattern appears in the other displayed previews and should be tested with explicit self-exclusion.

7.2 What the metrics do not measure

BERTScore measures similarity in an evaluator's representation, and ROUGE measures reference overlap. Neither guarantees that a generated holding, date or statute is legally correct. Strict NER scoring can reject a nearly correct entity because one character is misplaced. Conversely, a perfect schema does not make a prediction useful. The project needs both structural validation and task evaluation, followed by human legal review before any decision-making use.

The retrieval cutoff also has an intrinsic coverage limit. One displayed dev query has 92 relevant cases; even seven correct outputs would yield recall only $7/92 = 0.0761$ for that query. This does not explain away A's low overall recall, but it shows why the complete gold-set size must remain visible. I do not average the seven phase scores into one number: they measure different objects under different scales.

7.3 Limitations and what I would do next

A is the strongest recorded model, but its representation of long cases still depends on narrow selected windows. The immediate retrieval follow-up is a longer training budget with refreshed negatives, dev-best selection and a named baseline-preserving comparison. Its planned 30 epochs could raise the 817-query, batch-32 schedule from 78 to about 780 updates. No improved score from that proposal is reported here.

For B, the next test should first pass attention masks correctly and compare pretrained models at matched continuation-tail loss or training budget. For generation, repeat controls and a target-length study should precede a larger backbone. For LSI, threshold calibration and complete per-label support/F1 should be logged. For PCR, excluding non-relevant self-matches is a direct implementation ablation. These are specific tests of the observed failures, not claims that they will necessarily improve hidden-test scores.

7.4 Surprising findings

Parameter efficiency did not reliably preserve task quality in this record. A's judgment LoRMA result is far below full adaptation, although it changes much less of the model. Summarization gives a more useful counterpoint: A's LoRA run uses about 1.14% trainable parameters and remains the best recorded family. The contrast suggests that adaptation quality is task- and setup-dependent; the experiments do not support a blanket claim that low-rank updates replace full adaptation.

8 Conclusion

The implemented system covers all seven task interfaces with a shared set of compact foundations and separate task adaptations. Its most defensible result is empirical: the extensively trained dense A checkpoint is the strongest recorded model across the seven phases. Its main remaining limitation is restricted document coverage. B's poor results expose weaknesses in this particular training and feature-extraction setup, including a reproducible padding defect, rather than a general failure of encoder–decoder modelling.

The submission also records what remains unresolved: incomplete mandatory experiment coverage, unretained winning-run settings in early phases, repetitive summaries, self-matches in retrieval and the absence of human legal evaluation. Those limits determine how much can be concluded from the scores.

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor.

I used GPT (OpenAI) and Claude (Anthropic).

Q2. For each row, mark Yes or No and state the purpose in one short phrase.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Attention, residual accumulation, normalization and adaptation explanations.
Ideation / brainstorming at a high level	Yes	Pipeline organization, data checks and experimental follow-ups.
Debugging small ancillary snippets	Yes	Duplicate statute annotations and blank summary handling.
Plotting / visualization code	Yes	Original report plots, diagrams and evidence extraction.
Grammar / writing style of the report	Yes	Drafting and revision of the report prose.
Report structure / section wording	Yes	Organizing material under the supplied mandatory sections.
Designing the model architecture	No	Backbone/adapter implementations reported as my own; no backbone was generated in the recorded assistance.
Hyperparameter search / tuning	Yes	Longer A training recipes, candidate comparisons and automatic batch sizing, in addition to my own tuning.
Analysis of results / conclusions	Yes	Assisted numerical comparisons, failure diagnosis and discussion in this report.
Writing the core training / inference code	Yes	Assistance with task infrastructure, especially Phases 5–7 and retrieval batching/KD.
Other (specify)	Yes	Regression tests, packaging, source documentation and LaTeX compilation.

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them.

Yes. Sources include the course brief/template, official dataset and the architecture/adaptation papers listed in the bibliography: Qwen3, T5, gated DeltaNet, DeepSeek-V3, LoRA, LoRMA and QLoRA. Metric and implementation references include BERTScore, ROUGE, BLEU, knowledge distillation, PyTorch documentation, bitsandbytes and scikit-learn.

Q4. Apart from any permitted LLM use, what was your contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments and analysis. Name at least two things you tried that failed and what you concluded from them.

I proposed the overall project approach and worked on the model implementations. I chose experimental settings and carried out hyperparameter trials. I ran the training and submission workflow and supplied the recorded results.

Code repository. https://github.com/maheshwari-nitin/cs781

References

[1] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh, and A. Modi, "IL-TUR: Benchmark for indian legal text understanding and reasoning," *arXiv preprint arXiv:2407.05399*, 2024.

[2] CS781 Course Staff, "CS781 Capstone Document and Final Report Template," 2026. Course-provided assignment brief and Report_LLM.zip; IIT Kanpur.

[3] A. Yang *et al.*, "Qwen3 Technical Report," *arXiv preprint arXiv:2505.09388*, 2025.

[4] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.

[5] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[6] B. Zhang and R. Sennrich, "Root mean square layer normalization," *Advances in Neural Information Processing Systems*, 2019.

[7] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[8] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[9] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated delta networks: Improving Mamba2 with delta rule," *arXiv preprint arXiv:2412.06464*, 2024.

[10] DeepSeek-AI, "DeepSeek-V3 Technical Report," *arXiv preprint arXiv:2412.19437*, 2024.

[11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *International Conference on Learning Representations*, 2022.

[12] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," in *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 10113–10133, 2025.

[13] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," *Advances in Neural Information Processing Systems*, 2023.

[14] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in *International Conference on Learning Representations*, 2020.

[15] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, pp. 74–81, 2004.

[16] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pp. 311–318, 2002.

[17] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[18] bitsandbytes Contributors, “4-bit quantization: Linear4bit,” 2026. Accessed November 2026.

[19] scikit-learn Developers, “TfidfVectorizer api documentation,” 2026. Accessed November 2026.

A Evidence, submission ledger and requirement coverage

The evidence used here has three levels. The results document supplies screenshots of submission IDs, filenames, status and displayed scores, plus a rank beside each phase heading. Executed outputs in the later notebooks supply local scores, dataset counts, selected training messages and parameter statistics. Source files supply algorithms and default configurations. The last category is not treated as proof that an experiment ran. The source/evidence SHA-256 manifest records the original output-bearing notebook snapshots, before presentation cleanup. Numerical transcriptions are included under `report/evidence/`.

The 33 rows below are a transcription of the supplied screenshots, not a live leaderboard export. The submission `submission.zip` in Phase 1 cannot be assigned to a model family from its filename. The `B6f.zip` and `Aq6.zip` method mappings are supported by the corresponding saved notebook comparison, but exported submission-settings hashes are unavailable.

Table 8: All displayed submission attempts. Status is transcribed exactly; non-finished scores are not validated results.

P	ID	File	Status	Score
1	965311	qwen_dense_best.zip	Finished	0.8273
1	965281	qwen_dense_best.zip	Preparing	—
1	964252	gdn_hybrid.zip	Finished	0.8144
1	963258	mla_moe.zip	Finished	0.8077
1	963212	gdn_hybrid.zip	Preparing	—
1	963147	encdec.zip	Finished	0.8016
1	963065	submission.zip	Finished	0.8114
2	977524	D_ner.zip	Finished	0.2918
2	977517	C_ner.zip	Finished	0.2984
2	977456	B_ner.zip	Finished	0.0006
2	977453	A_ner.zip	Finished	0.3703
3	992390	A_rr_2.zip	Finished	0.5146
3	991383	D_rr.zip	Finished	0.3891
3	991379	C_rr.zip	Finished	0.4092
3	991369	B_rr.zip	Finished	0.3410
3	991366	A_rr.zip	Finished	0.4837
4	1006453	B_cj_kd.zip	Failed	0.6176
4	1006421	A_cj.zip	Finished	0.7935
4	1006399	B_cj.zip	Finished	0.5914
4	1006398	A_cj_lorma.zip	Finished	0.5634
4	1006397	A_cj.zip	Cancelled	—
5	1016707	D_lsi.zip	Finished	0.0730
5	1016609	B_lsi.zip	Finished	0.0328
5	1016608	C_lsi.zip	Finished	0.0951
5	1016369	A_lsi.zip	Finished	0.3517
6	1028889	B6f.zip	Finished	0.7651
6	1028888	C6.zip	Finished	0.7680
6	1028887	B6.zip	Failed	—
6	1028886	Aq6.zip	Finished	0.8094
6	1028885	A6.zip	Finished	0.8153
7	1038512	C_pcr.zip	Finished	0.0732
7	1038510	B_pcr.zip	Finished	0.0549
7	1038509	A_pcr.zip	Finished	0.1855

A.1 What the visible record establishes

Table 9: Coverage audit. Scored uploads include the four Phase-4 submissions. Counts alone do not establish distinct adaptation techniques or successful mandatory KD. A dash in "KD evidence" is not a claim that no attempt ever occurred.

P	Scored	Adaptation / model coverage visible	KD evidence retained
1	5	A, B, C and D filename variants; one generic filename	Not a task requirement here
2	4	A/B/C/D; methods not recoverable from filenames	No executed KD record retained
3	5	A/B/C/D plus second A upload	No executed KD record retained
4	4	Full A; A/B LoRMA variants in local table	B KD ran locally; upload failed
5	4	A full, B LoRMA, C partial, D KD in local table	Smaller D student recorded
6	4	A LoRA/QLoRA, B LoRMA, C LoRA	Local comparison explicitly says KD=False
7	3	A full, B LoRMA, C partial	Local comparison explicitly says KD=False

The failure/cancellation states are material. Judgment submission 1006453 displays 0.6176 but is marked failed, so it cannot replace the best finished score. Summarization's failed `B6.zip` is followed by a finished `B6f.zip`. Preparing rows in Phase 1 and the cancelled judgment row are similarly excluded. A minimum of four different methods cannot be inferred merely from four different architectures.

B Complete retained local comparisons

The following tables reproduce displayed numerical outputs, preserving local/competition separation.

B.1 Phase 4 local variants

Variant	Dev macro-F1	Trainable (%)
A full	0.814897	100.0000
B KD	0.640701	51.1502
B lorma	0.598750	0.5133
A lorma	0.562054	1.1428

Source: saved comparison output in `notebooks/30_cjpe.ipynb`. The exact run identifiers are:
`A_qwen_dense_full__full__last4x512`
`B_encdec__kd_from_A__last4x512`
`B_encdec__lorma__last4x512`
`A_qwen_dense_full__lorma__last4x512`

B.2 Phase 5 local variants

Variant	Macro-F1	Micro-F1	Subset acc.
A full	0.345304	0.492551	0.093311
C partial	0.097599	0.258762	0.036342
D KD	0.073143	0.235587	0.035360
B lorma	0.032393	0.141598	0.020332

Source: saved comparison output in `notebooks/40_lsi.ipynb`. The exact run identifiers are:
`A_full__spread4x512_3`
`C_partial__spread4x512`
`D_kd_from_A__spread4x512_3`
`B_lorma__spread4x512`

B.3 Phase 6 local variants

Variant	BERTScore	ROUGE-1	ROUGE-L
A lora	0.826998	0.301680	0.197140
A qlora	0.813496	0.278915	0.189584
C lora	0.765586	0.210960	0.171446
B lorma	0.762982	0.227685	0.192760

Source: saved comparison output in `notebooks/50_summ.ipynb`. The exact run identifiers are:
`A_lora__firstlast640_256`
`A_qlora__firstlast640_256`
`C_lora__firstlast640_256`
`B_lorma__firstlast640_256`All rows use 6,678 training documents and the same 32-document evaluation subset; no KD row is present.

B.4 Phase 7 local variants

Variant	Micro-F1@7	P@7	R@7
A full	0.192329	0.209443	0.177801
C partial	0.081156	0.088378	0.075026
B lorma	0.072262	0.078692	0.066804

Source: saved comparison output in `notebooks/60_pcr.ipynb`. The exact run identifiers are:
`A_full__firstlast2x256`
`C_partial__firstlast2x256`
`B_lorma__firstlast2x256`All rows use 817 training queries and 118 dev queries. The physical/effective query batch is 32 and embedding batch is 1,024; no KD row is present.

B.5 A quantitative comparison that is supported

In summarization, the two A adapter runs have the same displayed trainable count, 3,198,976, or 1.1392% of the original backbone. Their local BERTScore values differ by 0.013502. Their

competition scores, 0.8094 for QLoRA and 0.8153 for LoRA, also favor ordinary LoRA in this record. Complete per-run exports would still be needed to isolate the effect of quantization from other settings.

The judgment comparison does not establish successful parameter compression through KD. B has 284.483M original parameters, slightly more than A's 280.805M. Only B's feature-producing encoder and head receive gradients in classification, so the logged trainable count is 145.51M. The frozen decoder remains part of the model; the smaller trainable count does not establish a smaller complete student. In contrast, the LSI D backbone is about 233.974M, so its teacher-to-student transfer has a smaller total backbone as well as fewer trainable tensors.

C Dataset audit and retained distributions

Table 10: Recorded word-length quantiles. These marginal summaries do not recover histograms or paired compression ratios. Maxima are included because source selection must handle extreme cases.

Task	Split / field	Median	P90	P99	Maximum
LSI	train text	430	2,766	8,142	86,858
LSI	dev text	543	2,957	8,855	69,230
LSI	test text	532	2,811	8,306	134,848
Summary	train document	3,129	7,852	24,246	139,645
Summary	train reference	638	1,501	4,250	27,768
PCR	train query	4,508	12,602	42,997	119,513
PCR	train candidate	3,632	12,422	47,409	410,597
PCR	dev query	4,619	15,333	98,669	106,169
PCR	dev candidate	4,036	16,598	84,453	410,597
PCR	test query	4,653	14,064	28,981	91,838
PCR	test candidate	3,947	13,490	59,702	410,597

C.1 Statutes and relevance cardinality

The LSI record contains 42,750 train, 10,181 dev and 13,019 test documents. The train/dev union contains 100 IDs, with no dev-only label in the printed overview. The loader removes repeated annotations within a document before constructing multi-hot targets. This matters for frequency estimates and positive weights even though a binary target matrix would otherwise hide duplicates. Ten visible support counts are retained below; omitted rows are not imputed.

Statute	0	1	2	3	4	95	96	97	98	99
Train	343	1035	367	453	341	375	1973	7880	490	674
Dev	86	258	98	113	84	93	606	1970	123	176

PCR has 827 training queries and 4,320 candidates. Ten queries have no positive labels, leaving 817 usable supervised queries. The mean relevant count over all training queries is 6.418380; 248 have more than seven positives. Dev has 118 queries, 1,023 candidates, mean relevance 8.245763 and 39 queries with more than seven positives. Test has 237 queries and 1,727 candidates. No blank query/candidate text was reported in the displayed overview. Candidate IDs are allowed to recur across split files, but the pools are never merged.

C.2 Quality checks and evidence gaps

The summary loader excludes exactly the two recorded blank-reference IDs, 299 and 4799, leaving 7,028 usable examples. The recorded split is 6,678 training and 350 held-out documents. Its training-data fingerprint is retained in the saved notebook output; only 32 holdout IDs were generated for model selection. The grouping procedure hashes normalized document text so duplicate documents do not split between train and validation. A source-only check is not an empirical duplicate count, however; no such count was printed.

Phase-1 per-source counts, NER span-length/type histograms, and full CL/IT role frequencies were not retained in the supplied outputs. Nor were complete confusion matrices, per-label LSI F1 or all winning predictions. The code exposes these analyses, but the report does not substitute expected benchmark statistics for measured values. The additional evidence-export notebook can recover small metadata/log files from Drive without including model weights.

D Foundation specifications and objective details

D.1 A: dense decoder

For a residual state h, a block computes

$$\tilde{h} = h + \mathrm{GQA}(\mathrm{RMSNorm}(h)), \qquad h' = \tilde{h} + W_d[\mathrm{SiLU}(W_g\,\mathrm{RMSNorm}(\tilde{h})) \odot W_u\,\mathrm{RMSNorm}(\tilde{h})].$$

Query/key normalization precedes RoPE. There are 16 query heads and 4 KV heads, each with head dimension 64. The shared KV groups reduce stored keys/values relative to 16 independent KV heads. Attention uses causal scaled dot products; the cached single-step path attends to all existing keys. Feed-forward intermediate width is 2,816. The final normalization precedes a vocabulary head tied to the 32k embedding. These are implemented components of an inspired compact model, not a downloaded Qwen checkpoint.

The per-block count separates attention, MLP and normalization:

$$\begin{aligned} N_{\text{attention}} &= 1024^2 + 2(1024 \cdot 256) + 1024^2 = 2{,}621{,}440, \\ N_{\text{MLP}} &= 3(1024 \cdot 2816) = 8{,}650{,}752, \\ N_{\text{norms}} &= 2(1024) + 2(64) = 2{,}176. \end{aligned}$$

Thus each block has 11,274,368 parameters. Adding 22 blocks, a 32,768,000-parameter embedding and a 1,024-parameter final norm gives exactly 280,805,120 unique parameters. No separate untied output matrix is added.

D.2 B: encoder–decoder

B shares one 32k embedding between encoder input, decoder input and the LM output. Encoder layers use bidirectional attention; decoder layers add causal self-attention and cross-attention to encoder states. The default stacks contain ten layers each at width 1,024. During pretraining, the loader takes 768 source tokens and a 256-token target. Decoder inputs are BOS followed by the target shifted right. The loss is on target tokens only, so a 1,024-token window supplies one quarter as many directly scored positions as decoder-only next-token training. This is an objective difference, not a mathematical proof that the encoder–decoder needs exactly four times as many training steps.

For classification and retrieval, features come from the encoder alone. This has two consequences: the decoder no longer supplies useful task features, and the relevant depth/capacity is not

the full 20-layer parameter count. The code freezes tensors without a gradient, while preserving potentially inactive routed experts in other architectures. In summarization, both B stacks are trained; the classification shortcut must not be applied there.

D.3 C: gated DeltaNet/attention hybrid

C uses 24 layers, repeating three gated delta mixers and one gated softmax-attention layer. Its default MLP width is 1,920. The recurrent mixer is provided through `fla.layers.GatedDeltaNet`, with the correct `hidden_size` constructor argument and consecutive layer indices for cached state. A schematic gated delta recurrence is

$$S_t = \alpha_t S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top, \qquad o_t = S_t q_t,$$

with the library handling its normalization, convolution, gating and efficient kernel conventions. The remaining attention layers retain rotary positions and a sigmoid output gate. Kernel availability and exact dependency versions affect reproducibility.

D.4 D: latent attention and sparse experts

D uses width 768, 16 layers, query low-rank dimension 384 and KV latent rank 256. Its content-key/query dimension is 64, rotary component 32 and value head dimension 64, with 12 heads. The first layer has a dense feed-forward module. Later layers route each token to two of 12 experts and add one shared expert; expert FFN width is 384. Router bias affects top-k selection, while mixture weights are computed from the unbiased selected scores. Per-expert load buffers are accumulated in training and updated after an optimizer step. The optional multi-token head is a pretraining auxiliary and is not included in the summary-only or pooled-feature task losses.

D.5 Unequal foundation training

Loaded checkpoint metadata in the later notebooks gives A at step 16,800, B at 600 and C at 200. A's tail loss is 1.252103, B's 2.931996 and C's 2.972708. Exponentiating gives tail perplexities of approximately 3.50, 18.77 and 19.55. These diagnostics are more informative than comparing B's target loss with A's full-window loss, but they still do not create a matched-pretraining study. D's loaded latest checkpoint has no corresponding validation metadata in the record. The code's default global batch and context could be used to estimate tokens per step, but actual consumed tokens are not claimed here.

E Task losses, alignment and distillation

E.1 NER alignment and decoding

For each token offset (a_i, b_i), leading whitespace is trimmed while retaining coordinates in the original document. An entity $[s, e)$ assigns BIO labels to overlapping nonempty tokens; whitespace-only tokens strictly inside an entity can receive its I-label to avoid splitting it. The decoded span recovers the first and last token boundaries. Exact-match scoring therefore depends on representable boundaries. Nested/overlapping entities cannot all be encoded by one BIO sequence, and token-boundary differences may impose an oracle ceiling. The repository computes that ceiling; a numerical value is not available for this report.

For long documents, windows overlap and every token gets an owner. The causal owner maximizes left context; the bidirectional owner favors distance from the edges. Alternative mean aggregation averages overlapping-window predictions. Viterbi enforces legal BIO transitions but cannot recover missing text or repair a tokenizer boundary that is absent. No claim is made that either alternative was used for the winning NER upload because the final decoding export is missing.

E.2 Sentence roles and document classification

Rhetorical-role data already supplies sentence lists. The tokenizer records each sentence's interval in a concatenated stream, with a leading space before later sentences. Sentence intervals are pooled from their owner windows. Cross-entropy trains the role head; the final prediction retains the exact original sentence count. Judgment/LSI tasks instead pool token vectors into chunks and chunks into a case vector. Masking padded positions during pooling is necessary, but it is not equivalent to an attention padding mask inside B's encoder.

Binary judgment scores use two logits and cross-entropy. Macro-F1 averages class-specific F1 rather than weighting by the 59%/41% train prevalence. LSI applies a binary loss per statute and records macro-F1, micro-F1 and exact-set subset accuracy. The unweighted BCE observed in the saved recipe emphasizes many negative decisions; optional positive weights use training support only. Threshold calibration is implemented after fixed-threshold checkpoint selection. Calibrating on dev and reporting the same dev score is optimistic; fixed and calibrated scores are therefore saved separately.

E.3 Generation targets

For a causal model, the input concatenates selected document tokens and target-summary tokens. All next-token labels before the first summary token are -100. For B, source tokens are separate and the decoder predicts the summary using BOS-shifted inputs. Targets include EOT. A raw summary longer than the target budget is truncated before EOT is added; the generated validation metric still compares against the complete reference. This creates a known reference-length mismatch rather than an unrecorded change to the reference labels.

The Phase-1 continuation generator has a different contract: it targets a fixed 128 words and can ban EOT to avoid premature termination. Reusing that generator unchanged for summarization would be wrong. The summary generator permits EOT, bans BOS/PAD, supports sampling/repetition controls and can reuse the existing model cache. The recorded runs are greedy, with repetition controls inactive. BLEU is a secondary Phase-1 metric [16]; the summarization task's recorded secondary measures are ROUGE-1/L.

E.4 Distillation interfaces

The general temperature-scaled categorical signal is

$$\mathcal{L} = (1-\lambda)\mathcal{L}_{\text{gold}} + \lambda T^2 \,\mathrm{KL}\Big(p^{(T)}_{\text{teacher}} \,\|\, p^{(T)}_{\text{student}}\Big),$$

following the standard distillation formulation [17]. Judgment/NER/role teachers transfer class distributions when input alignment is compatible. LSI transfers independent sigmoid/Bernoulli targets instead of a softmax over statutes. Summary sequence KD can transfer generated text, permitting different tokenizers. Retrieval KD transfers similarity-score distributions aligned by candidate ID.

These code paths do not imply all required experiments were completed. The retained judgment KD did run and produce dev scores, but its submission failed. LSI has a completed D KD result. The summarization and PCR local comparison tables explicitly report no KD run. Cached retrieval KD requires fixed sampled candidate tuples; the later negative-resampling option rejects that incompatible configuration rather than silently misaligning teacher scores.

F Why B underperformed: implementation diagnostic and confounds

The source path `lm_lib.ner.model.hidden_states` calls `backbone.encode(ids)` for B without an attention mask. The B encoder consequently treats appended PAD tokens as visible positions. For a valid query token i, its attention denominator includes these extra keys:

$$\alpha_{ij} = \frac{\exp(q_i^\top k_j/\sqrt{d})}{\sum_{m=1}^{T+P} \exp(q_i^\top k_m/\sqrt{d})}.$$

Masking PAD states only after attention does not remove their earlier influence. A's causal attention, on the other hand, cannot attend from an original token to positions appended on its right.

Table 11: Fresh diagnostic, not a competition rerun. Seed 1337; vocab 32; width 32; two layers per stack; original IDs [3,4,5,6]; PAD ID 1. Relative change is measured on the original four positions.

Model	Added PAD	Relative L2 change	Max absolute change
A causal	4	9.07×10^{-8}	3.58×10^{-7}
A causal	8	5.77×10^{-8}	4.77×10^{-7}
B encoder	4	0.254340	0.670384
B encoder	8	0.313329	0.862638

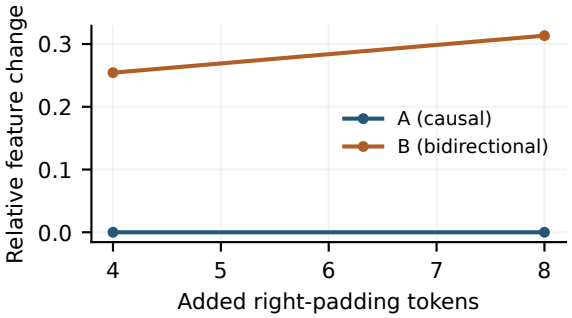


Figure 4: The same random-model implementation check. Competition-score attribution cannot be read from this curve. The full script and JSON output are included for reproduction.

The random-model check establishes a functional sensitivity in the implementation. It does not establish that 31.3% is the change in the trained B checkpoint, nor that fixing the mask would recover A's NER score. Padding is relevant to variable-length padded NER/role/classification batches. Summarization uses one document per forward and PCR inference groups equal-length

chunks, so those paths avoid this particular source of padding contamination. B's poor retrieval result still needs explanations beyond it.

The stronger global confound is the foundation itself. B has 600 recorded updates versus A's 16,800, and worse comparable tail loss. Its features come from ten encoder layers rather than A's 22 decoder layers, while its full parameter count includes a decoder unused in those tasks. A useful next experiment would first add the missing encoder padding mask, then match training budget or continuation-tail quality, and only then compare transfer. The report does not assert the outcome of that unperformed experiment.

G Concrete generation and retrieval errors

The summarization notebook prints generated examples as well as scalar scores. In the textile dispute example, the reference discusses the procedural effect of withdrawing a special-leave petition and the treatment of the court's order. The generated summary spends its budget repeating wage/dearness-allowance demands. This is a coverage failure visible in that example, not a quantified dataset-wide factuality rate. In the 229-word test preview for document 5994, one complete sentence is repeated 17 times. The source and derived repetition-count audit are retained as metadata without republishing the full raw documents.

Three displayed dev retrieval rankings begin with their own query IDs. For `0000035468`, the dev table records zero hits against 15 positives, and its ranking begins with `0000035468`. That first result is therefore a false positive for this query. The two displayed test examples likewise begin with their own IDs, but private test labels prevent computing their individual correctness here. The recorded retriever validates pool membership and uniqueness; it does not exclude query IDs.

Another displayed dev case, `0181443842`, has 92 relevant cases and only seven output slots. Its displayed hit count is three. This yields per-query recall $3/92 \approx 0.0326$, with a maximum of $7/92 \approx 0.0761$ under the required cutoff. The overall metric remains micro-averaged over complete gold sets, so high-cardinality queries materially affect the denominator.

No specific NER boundary mistake or named rare-role confusion is invented for this appendix. Raw error examples/confusion matrices for those phases were not saved in the supplied artifacts. The alignment and role-decoding limitations described elsewhere are implementation facts and hypotheses to investigate.

H Configuration provenance, execution and code layout

The submission has `code/phase1/` through `code/phase7/`, each with a selected-model source snapshot, a machine-readable training recipe, a provenance file, a phase README and a command-line launcher. `code/common/` holds shared libraries, all four architecture implementations, adapters, clean notebooks and tests. Keeping common dependencies separate prevents seven divergent copies of the same training code. The phase launchers insert that sibling common directory into the Python path.

For phases with no exported winning-run configuration, the recipe is explicitly reconstructed from the archived source/notebook. The provenance file distinguishes observed fields from defaults and unknown fields. Such a recipe reproduces the implementation, not necessarily the exact reported score. The second A rhetorical-role submission is a particular example: its winning settings cannot be inferred from the later notebook defaults. Recovering its original Drive config would remove that uncertainty.

For later phases, the final saved notebook cell can belong to another model (for example, D's LSI KD cell). It is not copied blindly as A's exact config. Snapshot metadata, run names, pool/chunk settings and comparison outputs identify the selected family; unknown A epoch/checkpoint choices remain labelled. The code supports loading the original run config supplied externally through the launcher.

The intended commands after installing the requirements and obtaining the official data are:

```
python code/phase5/run.py train --config code/phase5/recipe.json \
  --phase1-run /path/to/A_qwen_dense_full --data-dir /path/to/p5
python code/phase5/run.py predict --run-dir /path/to/lsi_run
```

Equivalent launchers cover the other task phases. Phase 1 additionally needs the tokenizer/bin preparation notebooks. Each phase README describes its required files and output schema. No model weights, checkpoints, token binaries, HF token, raw dataset or generated prediction dump is included. Environment authentication supplies dataset access; a token file is not part of the public code tree.

The selected official upload IDs are 965311, 977453, 992390, 1006421, 1016369, 1028885 and 1038509. Their filenames and four-decimal scores are in the ledger. Local run-to-archive mappings that are inferred from names rather than settings hashes are labelled accordingly in `selected_submission.json`. The code repository is https://github.com/maheshwari-nitin/cs781.

H.1 Checkpoint and interruption behavior

Phase 1 writes resumable optimizer checkpoints plus best/final weights. Later task loops save their best inference weights. Several newer loops protect nonempty output folders to prevent accidental overwrites; they do not implement optimizer resume. Therefore a new run name is required for a fresh fit. A deadline check happens between optimizer accumulation groups; final dev evaluation and checkpoint writing still take time after the deadline trigger.

Retrieval batching preserves the selected effective query batch through accumulation. Candidate sets remain independent within each physical batch, and embedding inference retries the same chunks with a smaller batch after a failed encoding attempt. These controls change execution batching without adding in-batch negatives or changing the retrieval objective.

H.2 Reproduction-sensitive details

The local summary scorer hash is

`roberta-large_L17_no-idf_version=0.3.12(hug_trans=5.22.0)`.

Its baseline-rescaling/IDF behavior and evaluator model need to match the competition before asserting identical scores. QLoRA uses bitsandbytes NF4 base projections [18], while C depends on FLA kernels. Exact historical dependency versions were not fully recorded.

I Unexecuted follow-up recipes and negative-mining safeguards

The longer A retrieval candidate is named `A_full__more_steps_v2`. It allows 30 epochs, LR 3×10^{-5}, 3% warmup, fresh epoch-seeded positive/negative subsamples and six-evaluation patience. The original context/temperature/effective batch are retained for this candidate. The optional

hard-negative and longer-context candidates change additional factors. They are not listed among measured results.

TF-IDF mining fits only the official TRAIN candidate text, using a sparse word/bigram representation [19]. All gold positives are excluded from negatives, even when a positive cap means some are not sampled into the current tuple. A mixture of hard and random negatives avoids training only on one static lexical neighborhood. Per-epoch tuples and the mined pool are saved. Queries with no lexical overlap fall back to random sampling. No dev/test relevance enters the training objective.

The A comparison helper checks the Phase-1 weight hash, train/dev fingerprints and actual query subsets. It chooses from the original measured A run and completed candidates, keeping the original on ties or regressions. The tuning notebook thus cannot label an unrun candidate as an improvement. Repeated dev selection still introduces selection bias; the report does not treat its chosen score as an unbiased generalization estimate.

J Verification record and unresolved evidence

The retained repository regression suite has 13 passing regression tests. It covers multi-label schemas and calibration, summary masking/shifting and EOT, retrieval pools/metrics, batching-gradient equivalence, negative exclusion, resampling, early stopping, checkpoint completeness and baseline-preserving selection. The report's additional padding diagnostic is separately labelled as a new random-model implementation check.

Before the final PDF was produced, both the notebook format and numerical transcription were checked, LaTeX was compiled, the ten-page main-body boundary was verified, and the archive was checked for its two required root directories and absence of weights/credentials. The report package retains sources so a reviewer can rebuild the PDF and figures. Remaining evidence gaps are exact early winning-run configs, full Phase-1 corpus/token logs and raw early-phase predictions.